Mapear agua de inundaciones con radar usando SAM

Guía del proyecto INF659 explicada desde cero, para presentarlo y responder preguntas · Noviembre 2026

EL PROYECTO EN UN PÁRRAFO

Queremos encontrar automáticamente dónde hay agua (ríos, lagos, zonas inundadas) en imágenes de radar del satélite Sentinel-1. Probamos un modelo de inteligencia artificial muy famoso, **SAM** (Segment Anything Model, de Meta), que sabe "recortar" objetos en fotos normales. Sin adaptarlo, falla en radar. Lo adaptamos paso a paso y, en datos que nunca usamos para tomar decisiones, llegó a un **IoU global de 0.642** en test y **0.657** en Bolivia, una región que ningún modelo vio al entrenar. Ambos superan la meta de 0.60. Como comparación honesta, una red simple (U-Net) entrenada desde cero logra 0.666 en test, pero 0.628 en Bolivia.

0.115	0.642	0.657	0.666
SAM sin adaptar (valid)	SAM adaptado, test (89 chips)	SAM adaptado, Bolivia (15 chips)	U-Net de referencia, test

Contenido

1. El problema y por qué usamos radar
2. El dataset Sen1Floods11, con un ejemplo
3. Cómo medimos si un mapa es bueno
4. Métodos clásicos: umbrales
5. Qué es SAM y cómo funciona
6. Primer experimento: SAM sin adaptar falla
7. El camino hasta 0.60, cambio por cambio
8. La U-Net: nuestra comparación honesta
9. Cómo evitamos engañarnos (protocolo)
10. Resultados finales
11. Comparación con otros trabajos
12. Límites, trabajo futuro y preguntas probables

1. El problema y por qué usamos radar

Cuando hay una inundación, los equipos de emergencia necesitan saber rápido qué zonas están bajo el agua. Los satélites ópticos (como fotos a color desde el espacio) no sirven cuando hay nubes, y durante lluvias extremas casi siempre hay nubes. Por ejemplo, en el Niño costero de 2017 en Piura, el cielo estuvo cubierto justo cuando más se necesitaban mapas.

El **radar** resuelve eso: el satélite emite su propia señal de microondas, que atraviesa las nubes y funciona de día y de noche. Usamos **Sentinel-1**, satélites gratuitos de la Agencia Espacial Europea.

CONCEPTOS CLAVE DEL RADAR

- **SAR (radar de apertura sintética):** el tipo de radar de Sentinel-1. Genera una imagen en escala de grises donde cada píxel dice cuánta señal "rebotó" de vuelta al satélite.
- **Retrodispersión:** esa señal que regresa. Una superficie rugosa (campo, bosque, ciudad) rebota en muchas direcciones y algo vuelve: se ve **clara**. El agua calma es como un espejo y rebota la señal lejos del satélite: casi nada vuelve y se ve **oscura**. Esa es toda la base física del proyecto.
- **Polarizaciones VV y VH:** el radar emite la onda en vertical (V) y la escucha en vertical (VV) o en horizontal (VH). VH es más sensible a la vegetación. Tenemos ambas bandas por cada imagen.
- **Decibelios (dB):** la señal varía muchísimo, así que se mide en escala logarítmica. Valores más negativos significan más oscuro. El agua suele estar en −20 a −30 dB en VH; la tierra, en −12 a −18 dB.
- **Speckle (moteado):** ruido granular típico del radar, como "sal y pimienta". Hace que píxeles sueltos de tierra parezcan agua y al revés.

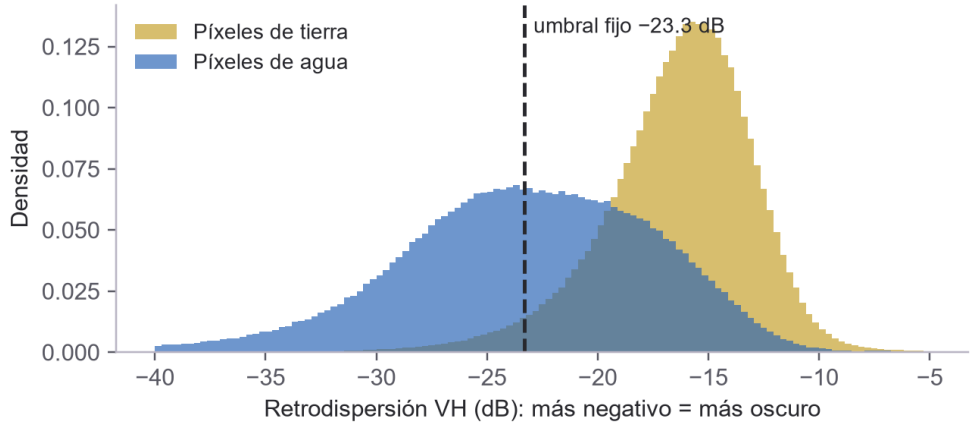


Figura 1. Distribución de la señal VH en ~1.3 millones de píxeles de entrenamiento. El agua (azul) es más oscura que la tierra (amarillo): sus medianas son −23.0 y −16.0 dB. Pero las dos campanas se **superponen**: por eso un simple umbral nunca es perfecto, y ahí es donde un modelo que mira el contexto (formas, bordes, vecinos) puede ayudar.

2. El dataset Sen1Floods11, con un ejemplo

Sen1Floods11 (Bonafilia et al., 2020) es el conjunto de datos estándar para este problema. Tiene imágenes de 11 inundaciones reales en 6 continentes. Usamos solo los **446 chips etiquetados a mano** por expertos (hay otros 4 385 con etiquetas automáticas, "débiles", que no usamos).

¿QUÉ ES UN CHIP?

Un recorte cuadrado de la imagen satelital de **512 × 512 píxeles**. Cada píxel mide 10 × 10 m, así que un chip cubre unos 5 × 5 km. Cada chip trae dos bandas de radar (VV y VH, en dB) y una **etiqueta** del mismo tamaño donde cada píxel vale 1 (agua), 0 (no agua) o −1 (sin etiqueta: nubes en la imagen óptica de referencia o zona dudosa). Los píxeles −1 no cuentan al medir.

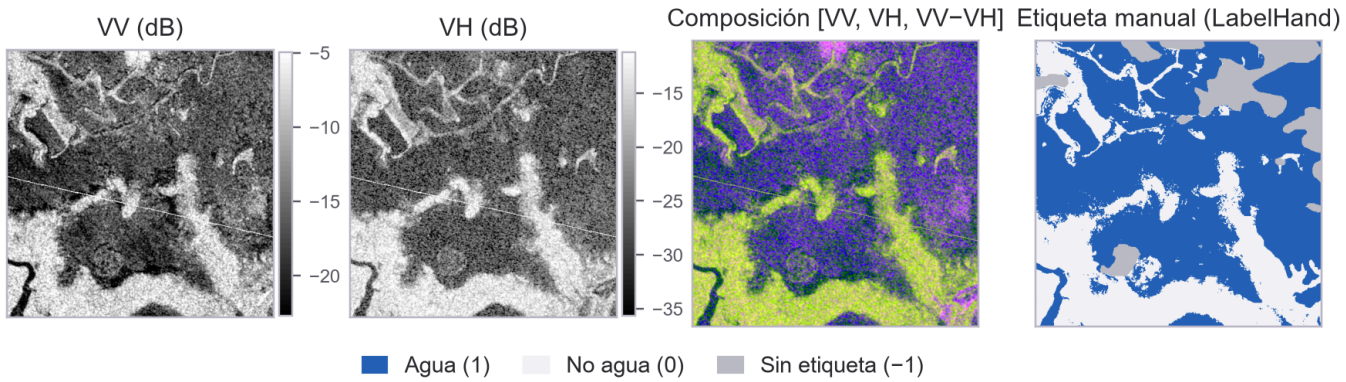


Figura 2. Un chip real de Bolivia (`Bolivia_129334`). En VV y VH el agua se ve oscura; la etiqueta marca 65.9 % de agua y 9.3 % sin etiqueta. En el centro, la composición de color que le damos a los modelos: rojo = VV, verde = VH y azul = VV − VH. Las zonas claras y verdosas son tierra; las oscuras y moradas, agua. La línea fina diagonal es un artefacto del sensor.

Cómo se divide el dataset

Partición	Oficial	Usamos	Para qué sirve
train	252	250	Entrenar: el modelo aprende viendo estos chips y sus etiquetas.
valid	89	86	Comparar variantes y tomar decisiones. Por eso sus números quedan algo optimistas.
test	90	89	Evaluación final, **una sola vez**. Son chips de las mismas 10 regiones que train.
Bolivia	15	15	Evaluación final en una región que **ningún modelo vio**: mide si el modelo generaliza.

Quitamos los chips con más de 95 % de píxeles sin etiqueta (2 de train, 3 de valid, 1 de test): casi no tienen nada que evaluar. Esa regla se fijó antes de ver resultados.

3. Cómo medimos si un mapa es bueno

Nuestro problema es de **segmentación binaria**: para cada píxel, el modelo responde "agua" o "no agua". Al comparar con la etiqueta, cada píxel cae en una de cuatro casillas:

- **TP** (verdadero positivo): dijo agua y era agua. **FP** (falso positivo o falsa alarma): dijo agua y era tierra.
- **FN** (falso negativo o agua omitida): dijo tierra y era agua. **TN**: dijo tierra y era tierra.

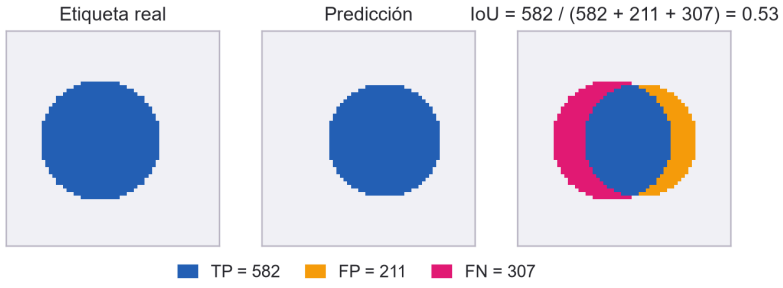


Figura 3. Ejemplo de juguete. El IoU (Intersection over Union) es el área donde predicción y verdad coinciden, dividida entre el área que cubren entre las dos. Vale 1 si el mapa es perfecto y 0 si no acierta nada. Los TN no entran, así que un mapa no gana puntos solo por decir "tierra" en todas partes.

LAS MÉTRICAS QUE USAMOS

- **IoU = TP / (TP + FP + FN).** Nuestra métrica principal, porque castiga por igual las falsas alarmas y el agua omitida.
- **IoU global** (la principal): se suman TP, FP y FN de *todos* los chips y luego se divide. Pesa más donde hay más agua.
- **IoU medio por chip** (complementaria): se calcula un IoU por chip y se promedia. Un chip casi seco pesa igual que uno inundado, así que sale más bajo (lo explicamos en la sección 10).
- **Precisión = TP / (TP + FP):** de lo que marcó como agua, cuánto lo era. **Sensibilidad (recall) = TP / (TP + FN):** del agua real, cuánta encontró.

¿CÓMO SABEMOS SI UNA DIFERENCIA ES REAL O SUERTE?

- **Bootstrap:** repetimos 5 000 veces el cálculo sobre un "sorteo con reemplazo" de los chips. El rango donde cae el 95 % de los resultados es el **intervalo de confianza (IC 95 %)**. Si el IC de una diferencia entre dos métodos no incluye el cero, la diferencia es confiable.
- **Pareado:** comparamos los dos métodos sobre los mismos chips sorteados, para no confundir "este método es mejor" con "le tocaron chips más fáciles".
- **Bootstrap por región:** igual, pero sorteando regiones enteras. Es más exigente, porque los chips de una misma inundación se parecen entre sí.
- **Prueba de Wilcoxon:** cuenta en cuántos chips gana cada método y si eso podría deberse al azar (valor $p < 0.05$ = poco probable que sea azar).

4. Métodos clásicos: umbrales

La forma más sencilla de hallar agua es elegir un valor de corte: todo píxel más oscuro que el umbral es agua. Probamos dos versiones:

- **Otsu por chip:** un algoritmo clásico (1979) que, mirando el histograma de *cada* chip, elige automáticamente el corte que mejor separa dos grupos. Después aplicamos una **apertura morfológica** (borra píxeles sueltos de "agua" causados por el speckle). Problema: si el chip casi no tiene agua, Otsu igual divide el histograma en dos y marca tierra oscura como agua. Resultado: **0.263** en test.
- **Umbral fijo + filtro de mediana:** un solo corte para todos los chips, **−23.3 dB en VH**, elegido mirando solo train (el que maximiza el IoU allí), después de suavizar cada píxel con la mediana de su ventana de 5 × 5 para reducir el speckle. Es mucho más fuerte: **0.582** en test. Lo tomamos de un repositorio público (*sar-flood-unet*) y reprodujimos su mismo valor.

OJO

Otsu es la base que usábamos al principio, pero es débil. La comparación justa es contra el umbral fijo. Lo agregamos **después** de abrir test, y así lo declaramos en todos lados.

5. Qué es SAM y cómo funciona

SAM (Segment Anything Model), de Meta (Kirillov et al., 2023), es un **modelo fundacional**: una red enorme preentrenada con 11 millones de fotos y más de mil millones de máscaras, que aprendió a recortar casi cualquier objeto. Su gracia es que es **promptable**: le das una pista (un punto, una caja o una máscara aproximada) y te devuelve el recorte del objeto señalado. Usamos la versión más pequeña, **ViT-B**, con 93.7 millones de parámetros.

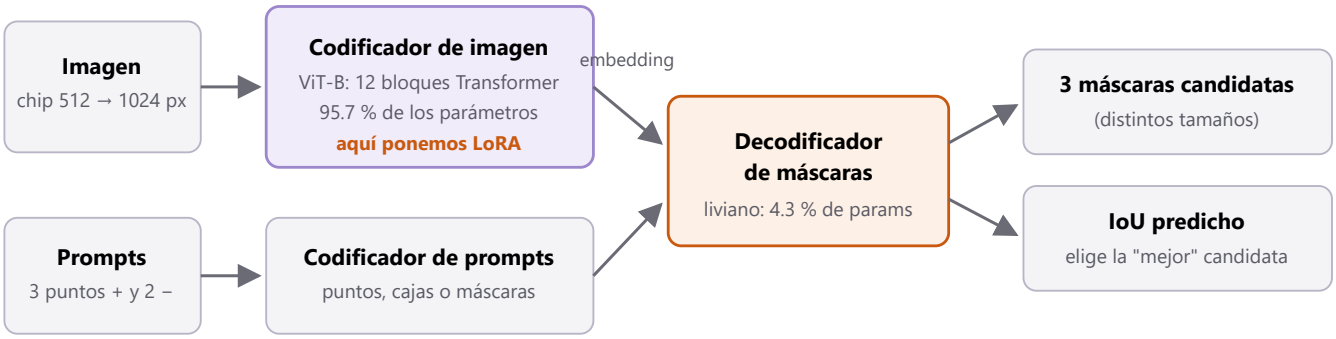


Figura 4. Arquitectura de SAM. El codificador de imagen es pesado (972 mil millones de operaciones por imagen) y se ejecuta una vez; el decodificador es liviano y puede ejecutarse muchas veces con distintos prompts. Por eso pudimos guardar el resultado del codificador ("embedding") y experimentar rápido con el decodificador.

CONCEPTOS DE REDES NEURONALES

- **Parámetros (pesos):** los números internos que la red ajusta al aprender. SAM ViT-B tiene 93.7 millones.
- **Transformer / ViT (Vision Transformer):** divide la imagen en cuadritos (parches de 16 × 16 píxeles) y deja que cada uno "mire" a todos los demás mediante un mecanismo llamado **atención**. Así entiende contexto: un píxel oscuro rodeado de un río probablemente es agua.
- **Embedding:** la "descripción" numérica que el codificador hace de la imagen: una grilla de 64 × 64 con 256 números por celda.
- **Preentrenado:** SAM ya aprendió con fotos normales; nunca vio radar. Ese es el gran reto: su conocimiento viene de un dominio muy distinto.

Prompts automáticos: nuestra idea de partida

SAM necesita prompts, pero no queremos que una persona marque puntos en cada imagen. Por eso los generamos **automáticamente con Otsu**: tomamos la máscara de Otsu, ponemos **3 puntos positivos** en el centro de sus manchas de agua más grandes (lo más lejos posible del borde) y **2 negativos** en la tierra. Las etiquetas nunca se usan para generar prompts, solo para evaluar.

6. Primer experimento: SAM sin adaptar falla (H1)

Nuestra primera hipótesis (H1) era que SAM, con esos prompts automáticos, mejoraría el mapa de Otsu. Pasó lo contrario: en valid, SAM sacó **0.115** frente a **0.256** de Otsu, casi igual que marcar todo el chip como agua (0.110). Ganó en solo 8 de 86 chips.

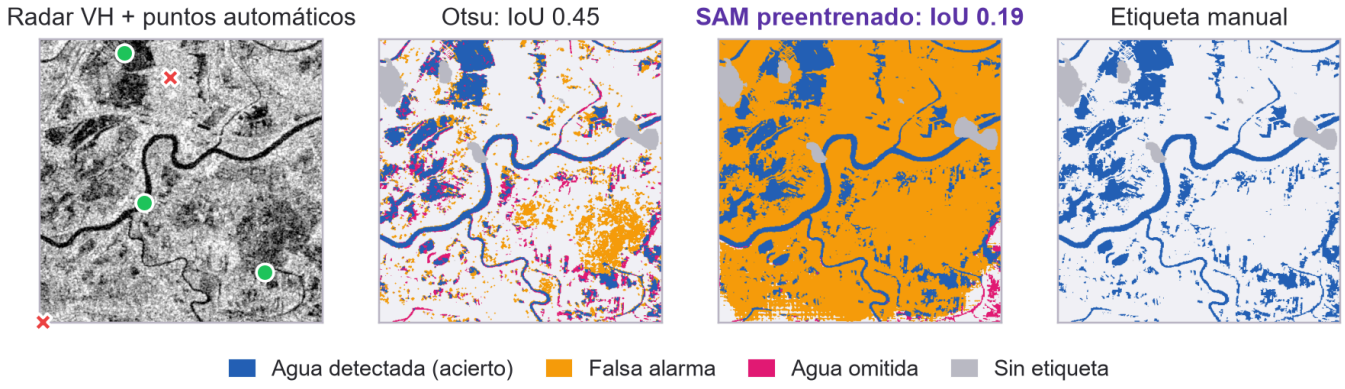


Figura 5. Chip de valid de India con 19 % de agua. Los puntos verdes son positivos y las cruces rojas, negativos. SAM marca el 95 % del chip como agua: casi todo es falsa alarma (naranja). Es el "colapso" que vimos en muchos chips.

¿Por qué falla? Los diagnósticos

- **Colapso:** en 41 de 86 chips, la máscara de SAM cubre más del 90 % del chip.
- **Su "medidor de calidad" no sirve en radar:** SAM elige entre sus 3 candidatas usando el IoU que él mismo predice. En radar, esa predicción no se relaciona con el IoU real (correlación de Spearman $\rho = -0.05$, prácticamente cero), y en 80 de 86 chips elige la candidata más grande.
- **No es culpa de los prompts:** con prompts perfectos tomados de las etiquetas (un "oráculo") llega a 0.234, y eligiendo siempre la mejor candidata, a 0.190. Ninguno supera a Otsu. Más puntos incluso empeoran: con 10 positivos y 10 negativos baja a 0.102.
- **La información sí está dentro:** una sonda lineal muestra que el codificador distingue agua de tierra (AUC hasta 0.934), aunque los dB crudos ya dan 0.906. El problema está en cómo se usa esa información.

CONCLUSIÓN DE ESTA ETAPA

No basta con darle buenos prompts a SAM: hay que **adaptarlo** al radar. Eso motivó todo lo que sigue.

7. El camino hasta 0.60, cambio por cambio

El profesor fijó como meta un IoU de 0.60. Llegamos en tres pasos; cada uno se planificó y registró antes de ejecutarlo.

¿QUÉ SIGNIFICA "ENTRENAR" O "AJUSTAR" (FINE-TUNING)?

- Se muestran a la red chips de train con su etiqueta. La red predice, una **función de pérdida** mide cuánto se equivocó y un **optimizador** mueve un poquito los parámetros para equivocarse menos. Repetir eso con todos los chips es una **época**.
- **Pérdida BCE + Dice:** BCE castiga cada píxel mal clasificado; Dice mide el solapamiento de las manchas (parecido al IoU) y evita que la red ignore el agua cuando es poca.
- **AdamW** es el optimizador; la **tasa de aprendizaje** (lr) dice cuán grande es cada pasito.
- **Semilla:** el número que fija el azar del entrenamiento (orden de los chips, inicialización). Entrenamos con 3 semillas para ver cuánto cambia el resultado por puro azar.

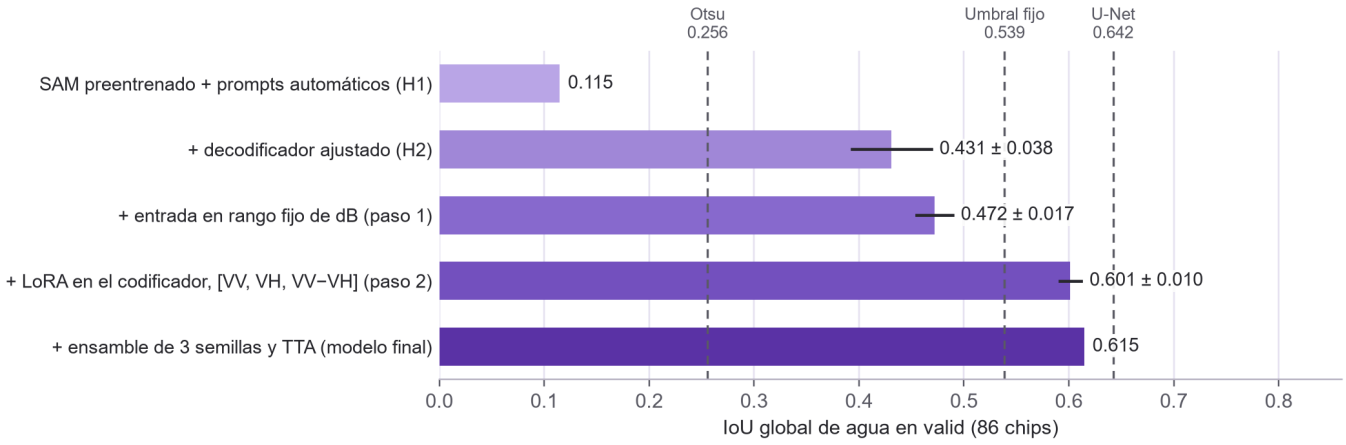


Figura 6. Cada barra agrega un cambio a la anterior (media ± desviación de 3 semillas). Las líneas punteadas son las referencias. Ojo: son resultados de valid, que usamos para decidir, así que están algo inflados; los números limpios son los de test y Bolivia (sección 10).

H2: ajustar solo el decodificador (0.115 → 0.431)

Congelamos el codificador (no cambia) y entrenamos solo el decodificador, que es liviano, durante 12 épocas. Como el codificador no cambia, su resultado se calcula una vez y se reutiliza: cada entrenamiento tarda minutos. Además pedimos una sola máscara de salida, para no depender del medidor de calidad que fallaba. Resultado:

0.431 ± 0.038, por encima de Otsu en las 3 semillas. Un control mucho más simple (una sonda lineal de 257 parámetros) se quedó en 0.073, así que el decodificador sí aporta.

Paso 1: normalizar la entrada en un rango fijo de dB (0.431 → 0.472)

SAM recibe imágenes con valores de 0 a 255. Al principio, cada chip se "estiraba" con sus propios percentiles (el 2 % más oscuro pasaba a negro y el 2 % más claro a blanco). Eso **borra el valor absoluto** de la señal, que es justo lo que distingue el agua: en un chip seco, la tierra más oscura terminaba pintada de negro, como si fuera agua.

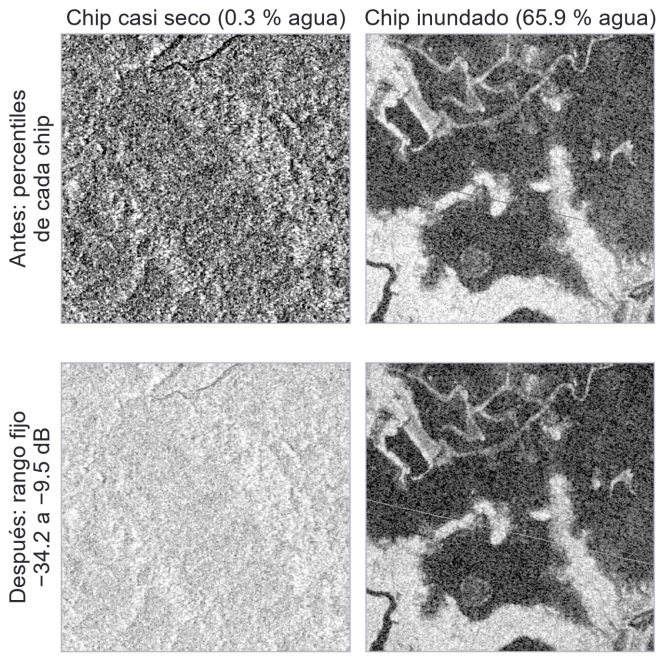


Figura 7. Arriba, el estiramiento por chip: el chip casi seco (izquierda) parece tener manchas oscuras. Abajo, el rango fijo (−34.2 a −9.5 dB para VH, calculado solo con train): el chip seco queda claro, como corresponde.

Con el rango fijo, un píxel de −25 dB se ve igual de oscuro en cualquier chip. Lo inspiró Garg et al. (2023), que recortan el radar a un rango fijo en dB.

Resultado: **0.472 ± 0.017**, contra 0.401 del mismo entrenamiento sin el cambio (+0.071).

En este paso también probamos dos cosas que **no sirvieron**:

- Usar las tres bandas [VV, VH, VV − VH] con el codificador congelado: 0.399, y muy inestable.
- Elegir la mejor época y el umbral con una parte separada de train ("train-dev"): sobrestimó valid entre 0.06 y 0.22 y ordenaba mal las variantes. Desde entonces fijamos todo de antemano.

Conclusión: con el codificador congelado ya no se podía mejorar mucho más. Había que tocar el codificador.

Paso 2: LoRA en el codificador (0.472 → 0.601)

Ajustar los 93.7 millones de parámetros con solo 250 chips es caro y arriesga que la red memorice train (**sobreajuste**). **LoRA** (Low-Rank Adaptation) es la solución estándar:

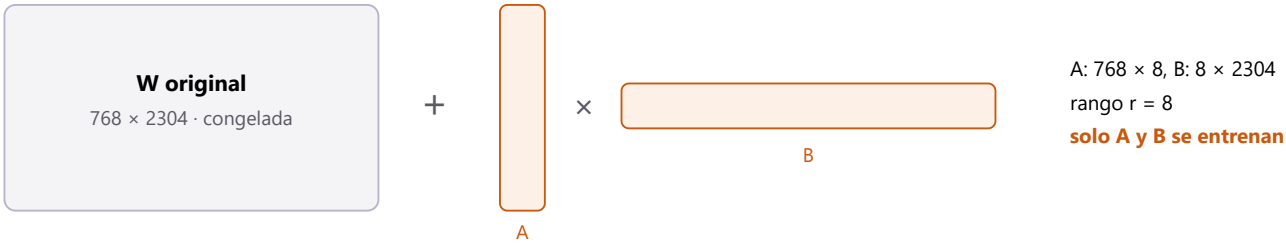


Figura 8. LoRA. En vez de modificar la gran matriz W de cada bloque, se le suma el producto de dos matrices delgadas (A y B). Con rango 8 son unos 295 mil parámetros nuevos en los 12 bloques; con el decodificador, 4.35 millones entrenables en total (4.6 % del modelo). B empieza en cero, así que al inicio el modelo es idéntico a SAM.

Además: entrada de 3 bandas [VV, VH, VV − VH], ahora que el codificador sí puede aprender a usarlas; 15 épocas, lr = 0.0001 y aumentos D4. Cada semilla tardó unos 16 minutos en una RTX 3070. Resultado: **0.601 ± 0.010**. Las 9 combinaciones probadas (3 variantes × 3 semillas) superaron con claridad al paso 1. Una variante **sin prompts** dio prácticamente lo mismo (0.602), señal de que, una vez adaptado, SAM depende poco de los puntos.

TRUCOS QUE SUMAN AL FINAL

- **Aumentos D4:** al entrenar, cada chip se rota (0°, 90°, 180°, 270°) y se voltea al azar. Una inundación sigue siendo inundación aunque se gire, y la red ve más variedad sin necesidad de datos nuevos.
- **TTA (test-time augmentation):** al predecir, el modelo mira las 8 versiones giradas o volteadas, se deshace el giro y se promedian las 8 probabilidades.
- **Ensamble:** se promedian las predicciones de los modelos de las 3 semillas. Los errores de azar de cada uno se compensan.
- **Umbral 0.5:** la red da una probabilidad por píxel; si es mayor que 0.5, es agua.

Con ensamble y TTA: **0.615** en valid. Este fue el modelo elegido para test.

Paso 3b, pérdida Tversky (no sirvió): el error principal era el agua omitida, así que probamos una pérdida que castiga más las omisiones. La sensibilidad subió (0.75 contra 0.70) pero la precisión bajó (0.77 contra 0.81), y el IoU (0.615 ± 0.008) no cumplió la regla de mejora fijada de antemano. Nos quedamos con el modelo anterior.

8. La U-Net: nuestra comparación honesta

Para saber si SAM realmente aporta, lo comparamos con una red clásica y mucho más simple: una **U-Net** (Ronneberger et al., 2015), el modelo de referencia en segmentación.

¿QUÉ ES UNA U-NET?

Una red convolucional con forma de "U". La primera mitad reduce la imagen paso a paso para entender el contexto ("¿qué hay aquí?"); la segunda la vuelve a agrandar hasta el tamaño original para decidir píxel por píxel ("¿dónde exactamente?"). Unos "atajos" (*skip connections*) pasan el detalle fino de la primera mitad a la segunda, para que los bordes salgan nítidos.

La nuestra tiene 7.76 millones de parámetros, se entrena **desde cero** (sin preentrenamiento) con la misma entrada, los mismos chips y los mismos aumentos, en unos 10 minutos por semilla. En valid sacó **0.637 ± 0.003** (0.642 con ensamble y TTA), mejor que SAM. Lo reportamos como hallazgo central, aunque no fuera lo que esperábamos.

9. Cómo evitamos engañarnos (protocolo)

- **Pre-registro:** antes de cada experimento escribimos la hipótesis, las configuraciones, lo que esperábamos y la regla para decidir si "funcionó". Así no podemos acomodar la conclusión después de ver los números. Está todo en `docs/bitacora_hacia_060.md`, con fecha y hora.
- **Test se abrió una sola vez** (5 de noviembre), con el modelo ya elegido antes de mirar. Si uno prueba muchas cosas en test y se queda con la mejor, el número sale inflado (la "maldición del ganador").
- **Sin búsqueda de hiperparámetros** en los pasos con LoRA: todo fijado de antemano, última época, umbral 0.5.
- **Desviaciones declaradas:** el chip de test `Paraguay_34417` no tiene datos de radar pero sí 50 026 píxeles de agua. Lo dejamos con predicción vacía para todos los métodos (todo su agua cuenta como error). Sin él, SAM daría 0.653 y la U-Net 0.677. El umbral fijo se agregó después de abrir test.
- **Uso de IA:** el diseño, el código y los pre-registros se prepararon con asistencia de Claude Code, y cada uso está anotado en la bitácora.

10. Resultados finales

Método	Test IoU	Test medio	Bolivia IoU	Bolivia medio	Precisión (test)	Sensibilidad (test)
Otsu por chip	0.263	0.235	0.467	0.427	0.300	0.676
Umbral fijo + mediana	0.582	0.314	0.613	0.371	0.812	0.673
SAM + LoRA (nuestro)	**0.642**	**0.405**	**0.657**	**0.438**	**0.836**	**0.734**
U-Net desde cero	0.666	0.421	0.628	0.397	0.814	0.785

Diferencia de IoU global [IC 95 %]	Test	Bolivia
SAM − umbral fijo	+0.059 [+0.026, +0.101]	+0.044 [−0.005, +0.092]
U-Net − umbral fijo	+0.083 [+0.053, +0.120]	+0.015 [−0.025, +0.057]
U-Net − SAM	+0.024 [−0.007, +0.061]	−0.029 [−0.069, +0.021]

Cómo leerlo:

- **SAM cruza la meta** en los dos conjuntos reservados: 0.642 en test (IC [0.510, 0.748]) y 0.657 en Bolivia (IC [0.477, 0.784]). Frente a Otsu, mejora +0.379 en test.
- **En test**, los dos modelos superan claramente al umbral fijo. La U-Net gana a SAM: el IC por chip toca el cero, pero el bootstrap por región y Wilcoxon (p = 0.011; gana en 50 chips contra 29) apoyan que la ventaja es real.
- **En Bolivia** el orden se invierte: SAM gana (Wilcoxon p = 0.046; 10 chips contra 3). Pero son 15 chips de una sola región y el IC incluye el cero: es una **señal**, no una prueba, de que el preentrenamiento de SAM generaliza mejor. Además, ahí el umbral fijo se acerca mucho a ambos modelos.

¿Por qué el IoU medio por chip es tan bajo (0.405)?

Casi un tercio de los chips de test (31 %) tiene menos de 1 % de agua. En esos chips, unos pocos píxeles mal marcados dejan el IoU cerca de 0 (promedian 0.15), pero en total solo aportan el 3 % de los píxeles. En el IoU medio cada chip pesa igual, así que lo hunden; en el global casi no pesan. A la inversa, 14 chips con más de 30 % de agua concentran dos tercios de los píxeles. Lectura: el modelo **mapea bien las inundaciones grandes**, pero todavía comete errores grandes, en proporción, en imágenes casi secas.

Estabilidad

Con 5 semillas en valid, SAM varía bastante más que la U-Net: 0.603 ± 0.017 contra 0.635 ± 0.004. En test, las tres semillas de SAM dieron 0.642, **0.553** y 0.642; la segunda falló sobre todo en un chip de Sri Lanka, y el ensamble corrige ese fallo. Por eso el modelo final es el ensamble.

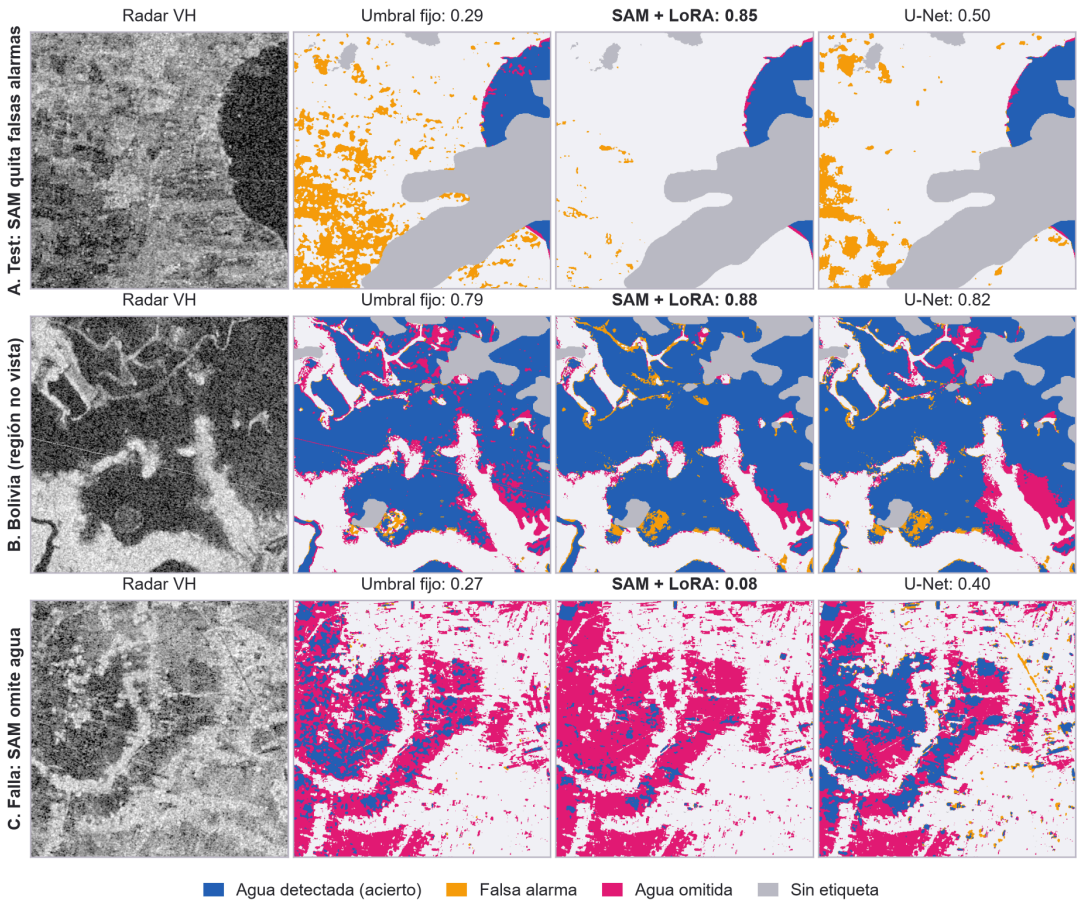


Figura 9. Tres chips elegidos por una regla fija (mayores diferencias entre modelos y más agua): son casos extremos, no típicos. El número es el IoU en ese chip. **A:** el umbral marca speckle como agua; SAM lo limpia. **B:** en Bolivia, SAM da el mapa más limpio. **C:** SAM omite la mayor parte del agua, mientras la U-Net encuentra bastante más.

11. Comparación con otros trabajos

Los números solo se comparan si usan el mismo dataset, la misma métrica y la misma entrada. Con radar solamente y etiquetas manuales de Sen1Floods11:

Trabajo	Qué hicieron	Su resultado	Nosotros
Bonafilia et al. 2020 (el dataset)	Red convolucional, etiquetas manuales; reportan IoU *medio por chip*	0.313 test	SAM 0.405, U-Net 0.421
Bonafilia et al. 2020, su mejor modelo	Con 4 385 chips de etiquetas débiles (17 veces más datos)	0.408 test	igualamos o superamos
Garg et al. 2023 (Google)	DeepLabv3+, solo radar, IoU global, a 16 m de resolución	0.676	SAM 0.642, U-Net 0.666
sar-flood-unet (repositorio)	U-Net con ResNet34 preentrenada; sin el chip sin radar	0.675 test, 0.576 Bolivia	SAM 0.653 / 0.657

- **Estamos en el rango de lo publicado** para solo radar con etiquetas manuales. Garg et al. llegan a 0.72 agregando un "maestro" que usa imágenes ópticas al entrenar.
- **DeepSARFlood (2025)** reporta 0.72, pero usa además elevación del terreno, mapas de agua permanente y miles de etiquetas propias: no es comparable.
- **Wang et al.** (preprint, sin revisión por pares) es el trabajo más cercano en método: SAM con LoRA en radar, y también concluyen que LoRA es lo que mejor funciona. Pero usan otro dataset (UrbanSARFloods) con una imagen previa a la inundación, así que sus cifras no se comparan.

- En una búsqueda no exhaustiva, **no encontramos un resultado publicado de SAM sobre Sen1Floods11 con solo radar**. Ese es nuestro aporte.

Exploración posterior (no cambia el resultado principal)

- **Mapa de agua permanente (JRC) como entrada extra de la U-Net:** casi no ayudó en test (+0.007) y **empeoró mucho en Bolivia** (−0.111 [−0.187, −0.046]). Nuestra hipótesis, sin probar: el modelo se "fía" del mapa y deja de marcar la inundación donde el mapa no indica agua. No lo recomendamos.
- **Preentrenar con etiquetas débiles:** se canceló antes de terminar; no hay resultado.

12. Límites, trabajo futuro y preguntas probables

Límites: Bolivia tiene solo 15 chips de una región; los intervalos son anchos (diferencias de 0.01 a 0.03 no se pueden afirmar con fuerza); valid se usó para decidir; SAM es inestable entre semillas; el decodificador de SAM produce máscaras de 256 × 256 que luego se agrandan, lo que podría limitar los bordes finos (hipótesis); y todavía no probamos datos de Perú.

Trabajo futuro: probar en la inundación de Piura 2017 contra los mapas de Copernicus EMS; usar información óptica solo al entrenar (como Garg et al.); y probar modelos fundacionales preentrenados con radar en lugar de fotos.

Preguntas que podría hacer el profesor

¿Por qué SAM, si al final una U-Net simple gana en test?

Porque la pregunta era si un modelo fundacional sirve para radar. La respuesta honesta tiene dos partes: dentro de las regiones de entrenamiento, una U-Net iguala o supera a SAM adaptado; en la región no vista (Bolivia), SAM fue mejor. Eso sugiere que su preentrenamiento ayuda a generalizar, aunque con 15 chips no lo podemos afirmar con seguridad.

¿El 0.60 es real o salió de elegir lo mejor en valid?

Es real: el modelo se fijó antes de abrir test, y test y Bolivia se evaluaron una sola vez. En valid sí elegimos, y por eso decimos que esos números (0.615) están algo inflados.

¿Por qué IoU global y no IoU medio?

El global mide qué tan bien se mapea el área inundada total, que es lo que importa en una emergencia. Siempre reportamos también el medio y explicamos por qué es más bajo (chips casi secos). Con la métrica del paper del dataset (IoU medio), también superamos su modelo con etiquetas manuales.

¿Qué es exactamente LoRA y por qué no ajustar todo SAM?

LoRA agrega matrices pequeñas que corrigen los pesos sin cambiarlos. Con 250 chips, ajustar 94 millones de parámetros arriesga memorizar train y no cabe cómodamente en una GPU de 8 GB; LoRA entrena el 4.6 % del modelo.

Si los prompts salen de Otsu, que es malo, ¿no arrastran sus errores?

Los puntos se ponen en el centro de las manchas más grandes, que suelen ser agua real. Además, la variante sin prompts dio casi lo mismo (0.602 contra 0.601): tras adaptarlo, SAM depende poco de los puntos.

¿Por qué el resultado de SAM varía tanto entre semillas?

Entrenar un modelo grande con pocos datos es sensible al azar: una semilla dio 0.553 en test por fallar en un chip grande. El ensamble de 3 semillas lo corrige; con más tiempo usaríamos 5.

¿Qué haría falta para pasar de ~0.65 a ~0.72?

Según la literatura, información extra: imágenes ópticas al entrenar o datos de terreno. Con solo radar y 250 chips, nuestros números ya están en el rango publicado.

¿Usaron inteligencia artificial para hacer el trabajo?

Sí, como asistente para diseñar, programar y redactar, y está declarado en la bitácora junto con cada decisión. Los pre-registros los debe revisar y firmar el equipo.

Fuentes de las cifras: `results/hacia_060/` (JSON de cada paso, test y robustez) y `docs/bitacora_hacia_060.md`. Figuras generadas desde esos resultados y desde los datos de Sen1Floods11.